

Safety Algorithm

Code: Write a C program to simulate: Producer-Consumer problem using semaphores

```
#include<stdio.h>

#include<pthread.h>

#include<unistd.h>

#include<semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

sem_t empty; // counts empty slots

sem_t full; // counts filled slots

pthread_mutex_t mutex; // for mutual exclusion

void* producer(void* arg) {

    int item, i = 0;

    while (1) {

        item = i++; // produce an item (here just incrementing
        number) sem_wait(&empty); // wait for empty slot
        pthread_mutex_lock(&mutex); // enter critical section
        buffer[in] = item;

        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&full); // signal that new item is available
        sleep(1); // simulate time to produce
    }

}

void* consumer(void* arg)
{
```

```

int item;

while (1)
{
sem_wait(&full); // wait for filled slot

pthread_mutex_lock(&mutex); // enter critical
section item = buffer[out];
printf("Consumed: %d from buffer[%d]\n", item,
out); out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex); // leave critical
section sem_post(&empty); // signal that a slot is free
sleep(2); // simulate time to consume
}

}

int main()
{
pthread_t prod, cons;

sem_init(&empty, 0, BUFFER_SIZE);

sem_init(&full, 0, 0); pthread_mutex_init(&mutex,
NULL); pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer,
NULL); pthread_join(prod, NULL);
pthread_join(cons, NULL);

sem_destroy(&empty);

sem_destroy(&full);

pthread_mutex_destroy(&mutex);

return 0;
}

```

Write a C program to simulate: Dining-Philosopher's problem

```
#include<stdio.h>
```

```
#include<pthread.h>
```

```

#include<unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork

pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;

    int left = id; // left fork index

    int right = (id + 1) % N; // right fork index

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);

        sleep(1); // Thinking

        // To avoid deadlock, philosophers with even id pick left then right, //
        odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork

            pthread_mutex_lock(&forks[left]);

            printf("Philosopher %d picked up left fork %d.\n", id, left); //
            Pick up right fork
            pthread_mutex_lock(&forks[right]);

            printf("Philosopher %d picked up right fork %d.\n", id, right);
        } else
        { // Pick up right fork

            pthread_mutex_lock(&forks[right]);

            printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up
            left fork pthread_mutex_lock(&forks[left]);

            printf("Philosopher %d picked up left fork %d.\n", id, left); }

        // Eating
    }
}

```

```

printf("Philosopher %d is eating.\n", id);
sleep(2); // Put down forks
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right); }
return NULL;
}

```

```

int main() {
int i;

int ids[N]; // Initialize forks (mutexes)
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}

// Create philosopher threads
for (i = 0; i < N; i++)
{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]); } //
Join threads (will never happen here, but good practice) for (i = 0; i
< N; i++)
{
pthread_join(philosophers[i], NULL); }

// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}

return 0;
}

```

}

Output:

```
C:\Users\Admin\Desktop x + v
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 25

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)  execution time : 37.861 s
Press any key to continue.
|
```

Deadlock Detection

Code: #include <stdio.h>

```
int main()
{
    int n, m, i, j, k;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m];
    int avail[m], finish[n], safeSeq[n];

    printf("Enter the allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter the request matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
```

```
{  
    scanf("%d", &request[i][j]);  
}  
}
```

```
printf("Enter the available resources: ");
```

```
for(i = 0; i < m; i++)  
{  
    scanf("%d", &avail[i]);  
}
```

```
for(i = 0; i < n; i++)  
    finish[i] = 0;
```

```
int count = 0;
```

```
while(count < n)
```

```
{  
    int found = 0;
```

```
    for(i = 0; i < n; i++)
```

```
    {  
        if(finish[i] == 0)  
        {  
            for(j = 0; j < m; j++)  
            {  
                if(request[i][j] > avail[j])  
                    break;  
            }  
        }
```

```
        if(j == m)
```

```

    {
        for(k = 0; k < m; k++)
            avail[k] += alloc[i][k];

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

int deadlock = 0;

for(i = 0; i < n; i++)
{
    if(finish[i] == 0)
    {
        deadlock = 1;
        printf("Process P%d is deadlocked\n", i);
    }
}

if(deadlock == 0)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

```



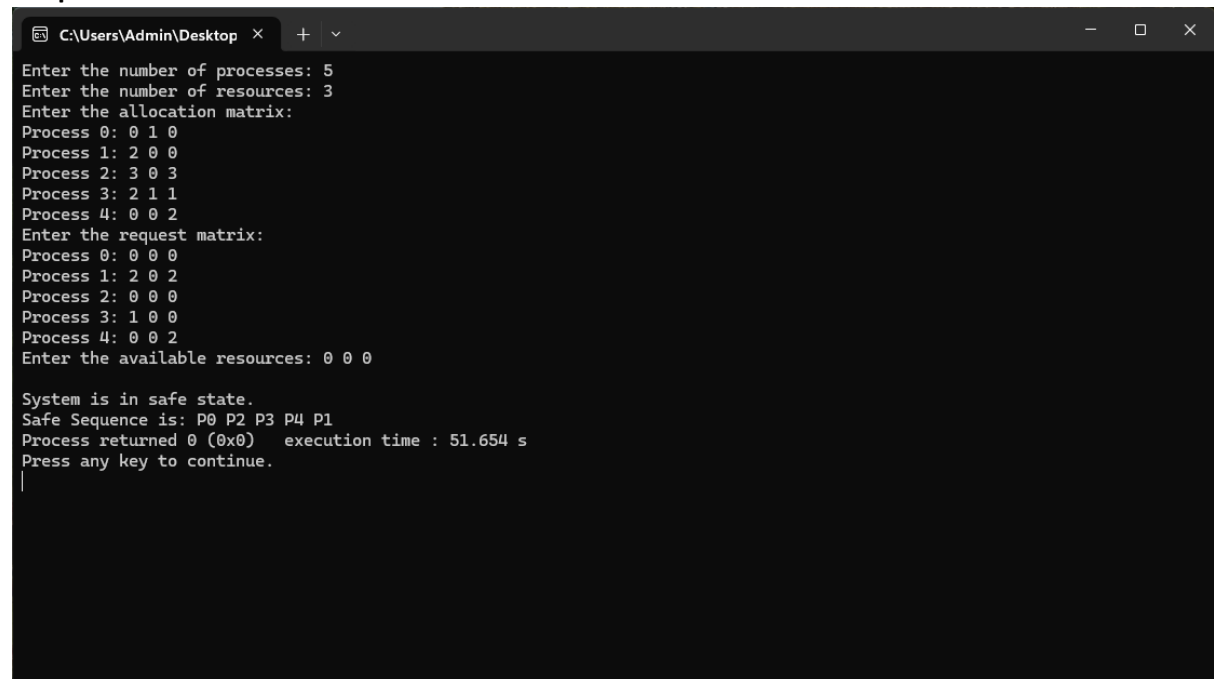
```

        for(i = 0; i < count; i++)
        {
            printf("P%d ", safeSeq[i]);
        }
    }

    return 0;
}

```

Output:



```

C:\Users\Admin\Desktop
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
Process returned 0 (0x0)   execution time : 51.654 s
Press any key to continue.
|

```